

Process Query Language: A Domain-Specific Language for Querying Event Logs of Business Processes

Tomasz P. Pawlak, Jędrzej Potoniec *

Abstract. Process Mining analyzes event logs from business processes to enhance understanding, verification, and improvement of these processes. Event logs typically originate from multiple sources and require extensive preprocessing before applying process mining techniques. This preprocessing includes filtering, composing, joining events into traces, and fixing or calculating attribute values. Existing process mining tools offer ad-hoc manual actions through graphical interfaces and simple domain-specific languages, but these methods have limitations, and there is no standardized approach for processing event logs. The primary contribution of this work is the Process Query Language (PQL), a domain-specific language (DSL) for processing event logs that surpasses current techniques. We begin by analyzing several existing DSLs concerning key success factors, followed by an evaluation of current business process DSLs to identify their limitations. Based on this analysis, we propose the PQL specification as a draft technical standard. PQL's design decisions are validated through an interpreter implementation and its application to numerous use cases.

Keywords: process mining, event log, data manipulation language, database, business process

1. Introduction

1.1. Motivation

Business processes are pervasive in today's economy: offices perform routine actions to serve customers, factories manufacture products on carefully designed production lines, and logistics services use standardized routes for goods delivery. Most business processes generate data as events recorded in computer systems. A business process

*Institute of Computing Science, Poznan University of Technology, Poland, {tpawlak, jpotoniec}@cs.put.poznan.pl

event log contains traces of cases, such as servicing individual customers or manufacturing specific product items. Each event within a trace represents a stage of an activity conducted by a resource, such as an employee or a machine.

Process Mining (PM) [65] is a maturing field focused on extracting knowledge from event logs. The main branches of PM include discovering process models from event logs, checking conformance between actual process execution and process models, and enhancing processes based on event distributions. For details on PM tasks, see, e.g., [10, 16, 20, 24, 42, 62, 74].

All these tasks depend on high-quality, well-structured event logs, where events are sequentially ordered by occurrence and have accurate attribute values. However, obtaining error-free event logs is challenging in practice, due to various deficiencies, such as:

- **Distribution** — Event data may be dispersed across multiple systems, some inaccessible due to different operators, resulting in incomplete logs.
- **Variety** — Event logs come in diverse formats, ranging from standardized structures like [4] (XES) to relational databases and unstructured flat files. Developing an Extraction, Transformation, and Load (ETL) pipeline for normalization and structuring is often time-intensive [21].
- **Missing order** — In distributed systems, establishing the exact sequence of events is challenging due to the absence of a shared clock [71]. The event order can only be approximated, especially when timestamps lack precision.
- **Scope** — Event logs may contain data across multiple views of the same process. For example, event order shows control flow, resources indicate executor communication, and numerical attributes (e.g., time, cost) serve as performance indicators. Each analytical question may require specific data, with excess information often complicating analysis.
- **Inadequate granularity** — Logged events may be too detailed or too generalized, making it difficult to answer specific process questions effectively.
- **Missing and concealed data** — Some necessary events or attributes might be absent and need to be derived from other event data.
- **Invalid data** — Some information may be inaccurate and require correction.

Consequently, event logs typically require extensive preprocessing before applying PM techniques. This includes composing logs from multiple sources, merging and splitting traces, grouping traces into similar variants, filtering to narrow the scope, merging events from different stages of the same activity, reordering events, and calculating attributes on the fly. For instance, in modeling a patient treatment process, logs must be compiled from multiple wards, traces merged per patient, and scope narrowed to a specific diagnosis. For a survey of event log preprocessing techniques, see [43].

There is no standardized approach for event log preprocessing tasks, and each PM software tool has its own methods. For instance, ProM [69] offers interactive

commands for individual tasks, RapidProM [66] provides a graphical editor for workflow processing, and Celonis [73] utilizes a simple domain-specific language (DSL) with interactive forms. Some academic studies [29,39] recommend using graph-based databases, such as Neo4J, to store event logs and query them using the Cypher language [30]. This approach may be challenging for users without technical experience, as Cypher was designed for querying graphs rather than event logs [39].

DSLs are gaining traction in both academic research and software due to their efficiency and simplicity for trained experts. Section 3 reviews DSLs developed for business processes thus far. However, none of these DSLs have become a widely adopted standard with implementations across diverse software tools. Most remain limited to narrow domains, lack comprehensive documentation, or use uncommon syntax.

1.2. Contributions

Based on this rationale, the goals of this work are as follows:

- Identify the key qualities of DSLs that drive successful adoption in technology (Section 2),
- Highlight the shortcomings of existing DSLs for business processes regarding these qualities (Section 3),
- Propose the Process Query Language (PQL) for event logs as a draft standard incorporating these features (Section 4 and Appendices A and B),
- Provide a reference implementation of the PQL interpreter,¹
- Assess whether PQL is sufficiently adequate and expressive to handle realistic event log processing use cases (Section 5).

Section 6 discusses the achievements of this study. Section 7 concludes this work.

2. Key qualities of successful DSLs

A DSL is a formal language tailored to a specific application domain, allowing concise expression of knowledge, formal objects, or tasks. Unlike General Purpose Languages (GPLs), DSLs are not suitable for tasks outside their specialized areas. However, the distinction between DSLs and GPLs is not absolute; they exist on a continuum, with highly specific DSLs like PCRE [35] at one end and broadly applicable GPLs like C++ and Java at the other [44].

In this section, we identify the key qualities that have led to the widespread adoption of certain DSLs in software development, such as SQL [8], HTML [36], CSS [19], JSON [56], and PCRE [35]. For example, SQL is reported by DB-Engines to be used

¹<https://github.com/ProcessMPUT/processm>

in seven of the top ten database engines, accounting for 72% of all database engines as of November 2024 [58]. Similarly, Forbes reports 1.09 billion websites as of October 2024 [34], most of which utilize HTML, CSS, and JSON. Additionally, PCRE and its reference library are widely used by software like Apache and nginx HTTP servers, and PHP and R interpreters.

DSLs are discrete formal entities, as are their properties. Dijkstra’s well-known quote [22], *In the discrete world of computing, there is no meaningful metric in which ‘small’ changes and ‘small’ effects go hand in hand, and there never will be*, is highly relevant here. Despite some past attempts, e.g., [37, 54], there are no objective, quantitative methods for assessing and comparing DSLs. Therefore, the key qualities we outline are categorical, and we provide conditions for meeting them rather than measures of satisfaction.

Based on the survey papers [44, 48, 55, 67, 72], we identify the following key qualities that make DSLs successful:

Grammar-based The language is grammar-based if its grammar is available in the machine-readable format. Textual DSLs utilize grammatically constrained syntax to ensure unambiguous language structure. This makes coding a highly efficient approach in software engineering and data mining applications, where the developer’s intention can be concisely expressed in just a few statements. For instance, in data retrieval, processing, matching, and visualization. As demonstrated in Section 3, coding is particularly effective for event log manipulation. In contrast, certain domains, such as robotics [48], use visual DSLs and low-code/no-code visual editors [27], however, these are intended mainly for crafting software solutions from larger off-the-shelf components and usually offer worse runtime performance than code-based approaches.

Declarative The language is declarative if it specifies the desired outcome of code execution rather than the procedural steps to achieve it. Many existing DSLs, such as SQL, HTML, CSS, JSON, and PCRE, are declarative. This approach enhances code readability and clarity, decouples the language from the interpreter’s implementation, and facilitates adoption across various platforms with differing capabilities. The guides for the development of DSLs [44, 72] recommend a declarative approach altogether.

High-level The language is high-level if it abstracts away from the technical aspects of implementation and directly manipulates domain objects. An SQL query specifies data retrieval, an HTML document defines user interface structure, CSS determines styling, JSON stores data structures, and PCRE expressions define text patterns. These languages do not dictate how data is retrieved, UI components are rendered, or patterns are matched. The distinction between high- and low-level languages is subjective, as higher-level languages can always be constructed atop others [72]. Here, we define a DSL as *high-level* if it enables direct manipulation of domain objects without exposing computational model details.

Focused DSL is focused if it defines domain-specific keywords to express desired results with minimal effort. One metric for this focus is the number of lines of code (LoC) in a DSL compared to a general-purpose language (GPL) for the same task, where GPLs typically require more code [72]. For example, SQL enables data manipulation in a single line, HTML and CSS are limited to defining user interfaces, and PCRE specializes in text pattern matching.

Complete DSL fully covers its application domain if it allows users to perform all relevant tasks with minimal effort. SQL offers extensive data manipulation capabilities, HTML and CSS provide flexibility in structuring and visualizing user interfaces, JSON enables storage of nearly any object-oriented data, and PCRE supports a wide range of regular expression patterns.

Portable DSL is portable if it avoids platform-specific features, allowing for consistent implementations across diverse platforms. SQL, HTML, CSS, JSON, and PCRE, for example, are supported on various operating systems and platforms.

Self-documenting DSL is self-documenting if the keywords and structures are closely aligned with domain objects, making the code resemble natural language statements that define or manipulate these objects. This often renders comments redundant, as they would merely restate what the code already conveys clearly.

Well-documented Language is well-documented if its every detail is thoroughly documented, including syntax, execution semantics, data model, error handling, null handling, and even number rounding rules. The DSL development guide [44] defines four levels of documentation quality for execution semantics, ranging from well-defined to non-executable DSLs. Only first-level DSLs provide consistent results across environments. SQL, HTML, CSS, JSON, and PCRE meet this highest level: the 2023 SQL standard spans 16 volumes, the HTML standard (as of writing) has 1,488 pages, CSS comprises over 100 modules, the 2017 JSON standard has 15 pages, and the 2016 PCRE specification is 46 pages.

Open Language is open if the documentation is freely accessible to all, e.g., on the Internet. Only open-access scientific papers and industrial standards are considered open. The HTML, CSS, and JSON standards, as well as the PCRE specification, are open. While most of the SQL standard is paid, database vendors provide free documentation of their implementations, effectively making much of [8] publicly available. This ensures that language details are accessible to both users and developers.

Familiar DSL is considered familiar if its syntax and keywords are similar to those in other languages, and the underlying data models adhere to technical standards, enhancing code readability. For the purpose of this study, we consider the DSL familiar if it closely relates to an existing language from top 20 programming languages as of 2024 [17]. Familiarity aids language learning and lowers entry barriers [55]. It

also supports the creation of derivative DSLs for other domains. For example, the SQL standard inspired numerous data manipulation DSLs, including JPQL [38], CQL [64], and EPL [28]. The HTML standard shares a common foundation in [1] with XML [75] and SVG [60]. JSON is also valid JavaScript. Many regular expression dialects share significant overlap with PCRE.

Supporting code quality DSL supports code quality if it is equipped with tools for code verification and static analysis. SQL, JSON, and PCRE interpreters immediately report code errors, while HTML and CSS validators detect errors through static analysis.

Maintainable DSL is maintainable if statements are not directly dependent on each other, e.g., do not share variable names. Examples are HTML tags, CSS rules, independent PCRE groups, and SQL clauses. This loose coupling facilitates code augmentation, as individual changes are mostly independent, ensuring correctness after each valid modification.

Practical use DSL has practical use if its every feature corresponds to a use case within its application domain demonstrated using an example in the documentation [48]. For practically usable DSLs, most keywords and structures are linked to multiple use cases, with reusable (boilerplate) code parts. Features without clear use cases clutter the language and impede implementation, portability, and maintainability, as emphasized by the WHATWG group in developing the HTML standard. DSLs with practically useful features also promote the reuse of software artifacts [44].

Scalable DSL is scalable if the length of code is linear or sublinear with respect to the number of objects it manipulates. Scalability, in terms of compilation time and system size, is also crucial [48].

For the reviews of notable DSLs proposed to date, see [48, 67].

3. DSLs for business processes

In this section, we review state-of-the-art DSLs for business processes and evaluate their properties with respect to the features identified in Section 2. The subsections cover DSLs for event log querying, complex event processing, querying process models, and other related works. Table 1 summarizes the features of the reviewed DSLs.

3.1. DSLs for event log querying

The works [31, 32] compare DSLs based on technical aspects and capabilities. They also introduce DAPOQL, a DSL that meets most (but not all) of the features defined in their study. DAPOQL is a declarative language that utilizes set algebra,

Table 1. Comparison of the qualities of the existing DSLs for business processes; ✓ stands for present quality, an empty cell for absent quality, and ? for the unknown status.

DSL	Grammar-based	Declarative	High-level	Focused	Complete	Portable	Self-documenting	Well-documented	Open	Familiar	Supporting code quality	Maintainable	Practical use	Scalable
DSLs for querying event logs														
DAPOQL [31,32]	? ¹	✓	✓	✓	✓	✓	✓		? ¹		✓		✓	✓
FPSPARQL [12]		✓	✓	✓	✓	✓	✓					✓	✓	✓
PIQL [,51]	✓	✓	✓	✓	✓	✓	✓		✓		? ²	✓	✓	
QUELI [11]		? ³	? ³	✓		? ³	? ³			? ³	? ³	? ³		? ³
XTemp [25]	✓		✓		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Celonis PQL [73]	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓
IQL [61]		✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓
PQL (this work)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
DSLs for complex event processing														
EPL [28]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
StreamInsight/LINQ [45]	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓
SEMEL [41]	✓	✓	✓	✓	? ⁴	✓	✓			✓		✓		✓
Model4CEP [15]		✓	✓	✓	✓	✓	✓		✓		✓	✓		
SAMZA SQL [47]	✓	✓	✓	? ⁵		✓	✓		✓	✓	✓	✓		✓
DSLs for querying process models														
Apromore PQL [52]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
BPQL [46]	✓	✓	✓	✓	? ⁶	✓	✓	? ⁶			? ⁶	✓	✓	✓
HIT [53]	✓		✓	✓	✓		✓	✓					✓	✓
APQL [63]	✓	✓	✓	✓		✓	✓		✓				✓	

¹ Grammar and complete documentation are not publicly available.

² As of writing, the PIQL tester, source code, and binaries are no longer available.

³ Many design details of QUELI are unpublished, and this project seems to be discontinued.

⁴ SEMEL documentation is incomplete and reference implementation is not available.

⁵ SAMZA SQL is more general and applies to not only to complex event processing but general SQL queries.

⁶ BPQL documentation and supporting software are proprietary and not available for closer assessment.

predicate-based filtering, temporal functions, multi-log schema, and more. However, the complete documentation for DAPOQL is no longer available as of the writing of this work.

FPSPARQL [12] is a DSL for querying event logs that extends a graph-query language SPARQL [59]. FPSPARQL employs a graph of events as the underlying data model and the specialized nodes called folders and paths for grouping events and storing the paths between them. The documentation of FPSPARQL is provided in natural language and lacks the level of formality guarantying unambiguity, e.g., it lacks grammar and even the math-based formulation of semantics.

PIQL [51] is an extension of the Decision Model and Notation (DMN) standard [23] for extracting key performance indicators (KPIs) from event logs. PIQL enables the building of conditions for decision points in business process models that depend on the KPI calculated using other business process cases. PIQL code resembles natural language and thus is easy to interpret.

QUELI [11] is an event log processing language aimed at detecting imperfections in event logs. This work does not provide the actual language specification but some building blocks for QUELI, such as the functions dedicated to the common tasks in deficiency detection, e.g., finding simultaneous events, finding partial order relations for activities, getting activity labels from events, detecting duplicate names.

XTemp [25] is an XSLT and XPath-backed DSL for processing event logs in an XML format, e.g., XES [4]. It employs the standard XML processing tools, e.g., XSLT processor, to execute the code. XTemp is well-documented using the OASIS standard [76]. It is feature-rich but typically requires large code snippets (tens of lines) to perform simple tasks. XTemp best suits the environments oriented on XML processing.

Celonis PQL [73] is a mature language for querying event logs with over 150 process-related operators. It is designed based on the needs of the actual business users. To date, Celonis PQL gained adoption by thousands of users of the Celonis platform. It uses a snowflake data model for representing event logs. Its reference implementation is column-store in-memory database. Although mature, not all technology details are publicly available, e.g., grammar.

IQL [61] is designed for querying incidents within event logs, where an incident is a set of activities in a temporal relationship matching certain patterns. IQL shares syntax with SQL and extends it with custom clauses and operators designed to describe operations on sequences of events. The same work provides theoretically grounded algorithms for the efficient execution of IQL queries. The reference implementation of the query engine appears to be incomplete, e.g., it lacks a language parser, which may hinder its practical use.

3.2. DSLs for complex event processing

Complex Event Processing (CEP) is a category of tools for processing event streams. Contrary to event logs, an event stream does not explicitly split into traces. However, the individual events may still hold the trace identifier. Some CEP tools come with

a CEP query language.

Esper EPL [28] is feature-rich comprehensive language for CEP. It is inspired by SQL-92 and supports time windows, filtering, aggregations, etc. EQL comes with the open-source Esper tool for efficient processing of the event streams in nearly real-time.

StreamInsight [45] is an add-on to the SQL Server. It uses the LINQ DSL, a subset of the C# programming language, to build event stream processing pipelines. It used to be a fully-featured commercial product with comprehensive documentation, however, it is discontinued.

SEMEL [41] is an SQL-like and ontology-based DSL for event processing. The language is documented incompletely and it is difficult to clearly say whether SEMEL is the DSL for querying event logs or querying event streams. SEMEL was not implemented in software according to [41].

Model4CEP [15] is a graphical DSL for meta-modelling CEP processing pipelines. Model4CEP enables the user to visualize virtually any CEP pipeline and convert it into one of the textual CEP languages, e.g., EPL [28]. Model4CEP comes with the editor [14] that supports code quality. The original article seems to not discuss all details, and the language is not comprehensively defined elsewhere. The graphical diagrams of Model4CEP may quickly grow for large pipelines. The documentation of the editor (as of writing) shows only a single use case.

SAMZA SQL [47] is an SQL-based DSL for complex event processing. It is limited to simple stateless queries including selections and projections. Its documentation is incomplete.

3.3. DSLs for querying process models

Another related group of DSLs is devoted to querying databases of process models based on their syntactic and behavioral features.

Apromore PQL [52] seems to be the most feature-rich and completely documented DSL in this field. Apromore PQL is a declarative DSL inspired by SQL. It is well-documented, comes with a reference implementation in the Apromore software [40], supports set algebra and predefined functions for assessing the relationships between activities. Although Apromore PQL shares the SQL-based syntax with the PQL proposed in this work, these two languages serve different purposes: Apromore PQL queries process models and PQL queries event logs.

BPQL [46] is a DSL for querying the state of a commercial workflow system. The complete documentation of BPQL is not publicly available, however, that workflow system seems to stay on the market for almost 20 years.

HIT model [53] is a framework for real-time processing of a stream of events raised by a workflow system. It operates on events but not on event logs. HIT is intended for supporting process monitoring rather than retrieval of model-related or event-log related data from a database.

APQL [63] is a DSL for selecting from a repository process models that fulfill certain conditions, e.g., relationships between activities. APQL is documented using abstract grammar and formal definitions of certain keywords, however, it is limited

to queries involving activity names and does not support other attributes.

3.4. Other related work

Relational XES (RXES) [70], a relational database schema for storing event logs. It is backward compatible with XES [4] and thus every valid XES event log can be stored in RXES. The experimental evaluation shows that the RXES implementation has a much smaller memory footprint than two state-of-the-art implementations of XES but is significantly slower in accessing event data.

The work [18] proposes a framework to query the active and past instances of workflows for data provenance. It defines a data model and some exemplary SQL statements for querying the status of long-running scientific experiments in a cloud.

4. Process Query Language

4.1. PQL at a glance

This section presents an executive summary of the design and features of the *Process Query Language* (PQL), the primary contribution of this study. The complete PQL specification is provided as a standard draft in Appendix A, while its grammar is detailed in Appendix B.

PQL adopts a data model compliant with the IEEE 1849 standard (XES) [4] for both source data and query results. PQL queries operate on a database of XES event logs and return (sub-)logs in the XES format.² It distinguishes the scopes of log, trace, and event, with optional attributes available at each scope. Attributes defined in [4] and its standard extensions are predefined in PQL with identical semantics, while other attributes can be used but lack defined semantics at the language level.

PQL employs a simplified SQL-like syntax, allowing optional clauses and permitting an empty query, which returns all event logs in the database unchanged. It supports arithmetic and logical expressions, aggregation functions, and dynamic grouping of events into traces. The below listing shows the simplified structure of a PQL query:

```
1 select <scope1>:<attribute1>, <scope2>:<attribute2>, ...
2 where <scope3>:<attribute3> = "value"
3 order by <scope4>:<attribute4>
```

The elements in angle brackets <> are placeholders. **scope#** refers to **log**, **trace**, and **event** prefixes for the identification of the entities whose attributes are referred to by **attribute#** names. All clauses are optional and can be omitted, in which case they do not constrain the resulting event log: without projection using the **select** clause,

²The event logs are not restricted to the XES XML interchange format but must adhere to the logical structure of logs divided into traces and traces into events.

all attributes are returned; without filtering using the **where** clause, all logs, traces, and events are returned; without ordering using the **order by** clause, all entities are returned in the same order as recorded in the original event log. Section 5 provides example PQL queries designed for specific use cases.

We implemented a PQL interpreter as part of the ProcessM system [50]¹ using the Antlr4 framework [49] in the Kotlin programming language [26] and the PostgreSQL database system [57]. The demo is available as a software-as-a-service (SaaS).³ This implementation allowed us to validate the design decisions behind PQL and address all identified shortcomings before publishing the initial specification. The system is expected to evolve as new features are added to the language.

4.2. Alignment with key qualities of success

PQL incorporates all key qualities of success identified in Section 2. PQL is *grammar-based* and its context-free grammar is given in Appendix B. PQL is *declarative*, as it declares the shape of the resulting event log rather than the way to obtain the result. It is *high-level* by abstracting from technical details of data processing and at the same time, it is focused on the domain of event logs compliant with [4]. PQL is *complete* as it enables concise expression of most typical use cases. It is *portable* and independent of specific platforms, database models, programming languages, or tools. PQL is *self-documenting* as it uses English keywords and natural language-like statements. Its details of semantics are *well-documented* in Appendix A. Its documentation is *open* and freely available both in this work and on the project repository.^{4,5} Thanks to the SQL-like syntax it aspires to be *familiar* to most software developers. It comes with a demo tool³ that verifies and executes PQL code and thus *supports code quality*. Individual clauses of PQL are largely independent and thus the code is *maintainable*. Section 5 shows several examples of *practical use*. The language is *scalable*, as the query size is independent of the size of the data being processed and returned and common keywords and standard attributes can be abbreviated.

4.3. Alignment with Cognitive Dimensions of Notations

PQL is firmly rooted in the theory of Cognitive Dimensions of Notations (CD) [13,33]. CD is a framework for creating notations, languages, and user interfaces that consider usability and ease of adoption. It outlines 14 independent quality dimensions and their trade-offs. Here, we summarize how PQL aligns with CD. PQL fulfills *closeness of mapping* by using event log-centric terms, standardized attribute names [4], and SQL-based query clauses [8]. It minimizes *error-proneness* through abstract data manipulation and keywords that align semantically with the domain. PQL's *abstraction* level is well-defined, as it directly interacts with event log structures, eliminating

³<https://demo.processm.cs.put.poznan.pl>

⁴<https://github.com/ProcessMPUT/processm/blob/master/docs/pql.md>

⁵<https://github.com/ProcessMPUT/processm/tree/master/processm.core/src/main/antlr4/processm/core/querylanguage>

the need for context definition. By standardizing structures for similar concepts, PQL enhances *consistency*, improving learnability and reducing cognitive load. *Role-expressiveness* is achieved via explicit clause naming for specific tasks. PQL queries avoid *premature commitment* by not requiring implementation-level choices, allowing users to express needs freely. It boasts low *viscosity*, needing minimal query edits for significant output changes. PQL design avoids *hidden dependencies*; all clauses are optional, and new clauses can be added to enhance output semantics, encouraging *progressive evaluation* and *provisionality* via incremental query refinement. With structured indentation and optional comments, PQL supports *secondary notation* for improved readability and collaborative query work. Queries are typically short, promoting *visibility* of components. Clear abbreviation rules for attribute names help minimize *diffuseness*. Finally, PQL lessens the need for *hard mental operations*, allowing users to specify output properties declaratively rather than detailing specific steps.

5. Assessment of PQL

This section addresses the research question: *How expressive is PQL?*

Quantitatively assessing a programming language’s expressiveness is inherently subjective. However, we calculate key performance indicators (KPIs) for PQL that correlate with its expressiveness. To evaluate, we define realistic event log processing use cases and provide handcrafted PQL queries to implement them. For certain use cases, multiple equivalent queries are presented to demonstrate the language’s flexibility.

To ground the use cases in a real-world context, we utilize the event log [68], which contains data from an academic hospital in the Netherlands. The use cases are grouped into subsections, starting with basic operations such as data retrieval, filtering, and projection, progressing to more complex scenarios.

Table 2 shows KPIs for each query:

- Total number of clauses, where clause is defined in Appendix A;
- Total number of terms, where term is a keyword, an attribute name, a literal, or an operator; comparison sign is not a term;
- Total number of characters excluding whitespace between terms and comments.

The majority of queries consist of a single clause, with only five out of 28 queries (across 17 use cases) involving multiple clauses, with a maximum of five clauses. All queries utilize at most 23 terms (median: 7) and a maximum of 145 characters (median: 50.5). These results demonstrate that PQL queries are concise while effectively defining complete processing pipelines.

Table 2. The KPIs for PQL queries implementing the corresponding use cases: the total numbers of clauses, terms, and characters.

Use case	Query	# clauses	# terms	# characters	Notes
UC1	Q1	0	0	0	
UC1	Q2	1	2	7	long form
UC2	Q3	1	3	36	long form
UC2	Q4	1	3	34	long form
UC2	Q5	1	3	26	
UC3	Q6	1	8	51	
UC4	Q7	1	9	108	long form
UC4	Q8	1	9	104	long form
UC4	Q9	1	9	100	long form
UC4	Q10	1	9	98	
UC5	Q11	1	9	71	long form
UC5	Q12	1	9	60	
UC6	Q13	1	9	72	long form
UC6	Q14	1	9	61	
UC7	Q15	1	2	22	long form
UC7	Q16	1	2	12	
UC8	Q17	1	4	20	
UC9	Q18	2	6	50	
UC10	Q19	1	10	120	long form
UC10	Q20	1	10	76	
UC11	Q21	1	3	26	long form
UC11	Q22	1	3	12	
UC12	Q23	1	3	18	
UC13	Q24	2	10	59	
UC14	Q25	1	3	13	
UC15	Q26	2	5	23	
UC16	Q27	5	20	124	
UC17	Q28	5	23	145	
Median		1	7	50.5	

5.1. Reading the entire database

UC1 In order to read the entire database, including all event logs, all their traces, and all their events in the recorded order use one of the queries:

```
Q1 /* The empty query */
Q2 select *
```

Queries Q1 and Q2 are functionally equivalent. Q1 omits all control keywords, returning all data without processing, while Q2 explicitly retrieves all attributes across all scopes, also without any processing.

5.2. Filtering event logs, traces, and events

UC2 In order to select all event logs named ‘Hospital log’, all their traces, and all their events in the recorded order use one of the queries:

```
Q3 where log:concept:name='Hospital log'
Q4 where l:concept:name='Hospital log'
Q5 where l:name='Hospital log'
```

Queries Q3 to Q5 are equivalent. Q3 uses the full form of attribute reference, where `log` is the scope prefix, `concept` is the standard extension prefix as defined in [4], and `name` is the standard attribute name. The `concept:name` attribute at the `log` scope refers to the human-readable name of the event log. PQL allows shorthand notation for scope and standard attributes from [4], reducing the scope prefix to its first letter, `l` in Q4 and Q5, omitting the attribute prefix in Q5.

UC3 In order to filter the results to the traces with diagnosis set:

```
Q6 where l:name='Hospital log' and [t:Diagnosis] is not null
```

The `t:Diagnosis` attribute, where `t` refers to the trace scope and `Diagnosis` to the attribute name, is non-standard and requires surrounding square brackets. The square bracket notation enables the use of non-standard attributes, accommodating virtually any symbols in their names, including whitespace.

UC4 In order to filter the results to the traces with diagnosis *maligniteit cervix* and events raised in the nursing ward:

```
Q7 where l:name='Hospital log' and [t:Diagnosis]='maligniteit cervix'
    and event:org:group='Nursing ward'
Q8 where l:name='Hospital log' and [t:Diagnosis]='maligniteit cervix'
    and e:org:group='Nursing ward'
Q9 where l:name='Hospital log' and [t:Diagnosis]='maligniteit cervix'
    and e:group='Nursing ward'
Q10 where l:name='Hospital log' and [t:Diagnosis]='maligniteit cervix'
    and group='Nursing ward'
```

Queries Q7 to Q10 are equivalent. The `event:org:group` term refers to the standard attribute `org:group` from [4] at the `event` scope. Q8 to Q10 use the shorthand notation: Q8 and Q9 reduce the scope prefix to `e` and Q10 drops it completely, since the `event` scope is the default; Q9 and Q10 drop the attribute name prefix too.

UC5 In order to select the events in the Hospital log that occurred on weekends and filter out events on workdays:

Q11 `where l:name='Hospital log' and dayofweek(event:time:timestamp) in (1,7)`

Q12 `where l:name='Hospital log' and dayofweek(timestamp) in (1,7)`

Q11 and Q12 are equivalent. The `dayofweek(t)` function converts the timestamp argument to the day of week, where 1 stands for Sunday and 7 stands for Saturday. Q12 uses the shorthand notation and drops the scope and attribute name prefixes.

UC6 In order to select the entire traces with any event that occurred on weekend and filter out the traces without events on weekends:

Q13 `where l:name='Hospital log' and dayofweek(~event:time:timestamp) in (1,7)`

Q14 `where l:name='Hospital log' and dayofweek(~timestamp) in (1,7)`

Queries Q13 and Q14 are equivalent. The hoisting prefix `~` shifts the scope of the succeeding attribute one level above. In both Q13 and Q14 the `time:timestamp` attribute, originally at the `event` scope is moved to the `trace` scope. Here, the `dayofweek()` function returns a vector of weekdays corresponding to individual events, and the `in` operator checks if any element of this vector equals 1 or 7. No filtering is applied at the event scope.

5.3. Projecting attributes

UC7 In order to select the names of all event logs in the database:

Q15 `select log:concept:name`

Q16 `select l:name`

Queries Q15 and Q16 are equivalent. Both select only the `concept:name` standard attribute at the log scope, while no attributes are selected for traces and events. As a result, traces and events are returned but contain no attributes.

UC8 In order to select the names of all event logs and all attributes of their traces and events:

Q17 `select l:name, t:*, e:*`

The terms `t:*` and `e:*` select all attributes within the trace and event scopes, respectively.

5.4. Calculating key performance indicators

UC9 In order to count the traces in the Hospital log:

Q18 `select count(^t:name) where l:name='Hospital log'`

The hoisting prefix `^` shifts the scope of the `concept:name` attribute from the trace scope to the log scope, enabling aggregation with the `count()` function. This function counts non-empty values of the specified attribute, thereby counting only the named traces.

5.5. Aggregating traces

UC10 In order to select the beginning and the end timestamps of the trace and calculate its duration:

Q19 `select l:name, min(^event:time:timestamp), max(^event:time:timestamp)`
`,`
`max(^event:time:timestamp)-min(^event:time:timestamp)`
`group by t:name`

Q20 `select l:name, min(^timestamp), max(^timestamp), max(^timestamp)-min`
`(^timestamp)`
`group by t:name`

Queries Q19 and Q20 are equivalent, with Q20 employing shorthand notation. The hoisting prefix `^` elevates the scope of the `time:timestamp` event attribute to the trace scope, where the `min()` and `max()` functions identify the smallest and the largest timestamps of the events within a trace, respectively. Subtracting these timestamps computes the number of days between them.

UC11 In order to group the traces into variants, where traces belong to the same variant if their sequences of event names are equal:

Q21 `group by ^event:concept:name`

Q22 `group by ^name`

5.6. Reordering events

UC12 In order to order events using their timestamps rather than the recorded order:

Q23 `order by e:timestamp`

UC13 In order to order traces descending using their duration:

Q24 `group by t:name order by max(^e:timestamp)-min(^e:timestamp) desc`

5.7. Limiting results

UC14 In order to restrict the output to the top 5 event logs and the top 10 traces per log.

Q25 `limit 1:5, t:10`

UC15 In order to read the next 10 traces in the top 5 event logs:

Q26 `limit 1:5, t:10 offset t:10`

5.8. Combining all together

UC16 In order to retrieve the top 30 most common trace variants in the Hospital log based on activity names, along with the total number of traces, total occurrences per variant, and activity names:

Q27 `select l:name, count(~t:name), count(t:name), name
 where l:name='Hospital log'
 group by l:name, ^name
 order by count(t:name) desc
 limit t:30`

UC17 In order to retrieve the top 30 longest traces in the Hospital log, their names and duration, and all attributes of their respective events:

Q28 `select l:name, t:name, max(~timestamp)-min(~timestamp), e:*
 where l:name='Hospital log'
 group by t:name
 order by max(~timestamp)-min(~timestamp) desc
 limit t:30`

6. Discussion

Evaluating formal languages inherently presents challenges due to the absence of objective quantitative measures for assessing their quality and suitability for a given problem domain. The formulation of the key qualities in Section 2 draws on insights from prior work [44, 48, 55, 67, 72]. These qualities do not guarantee the success of a language; instead, they represent characteristics commonly found in widely adopted DSLs that have become de facto standards in their respective domains. This list is not comprehensive, as additional qualities addressing other facets of language usage and application undoubtedly exist.

Determining whether a DSL satisfies a key quality can be inherently subjective, as factors such as user familiarity or perceived maintainability depend on context and perspective. To reduce subjectivity, we established explicit criteria for each quality to consistently evaluate the languages surveyed in Section 3.

For example, familiarity is strongly influenced by the user’s background and exposure to specific technologies. FPSPARQL [12] and PQL (Section 4) both employ industry-standard syntax — FPSPARQL is based on SPARQL [59], and PQL is derived from SQL [8]. While SQL syntax is widely recognized and familiar to a broad audience of developers, SPARQL is niche, as evidenced by its exclusion from prominent programming language surveys like [17].

Completeness similarly depends on the assumed boundaries of a DSL’s domain. A language addressing a broad domain is more likely to be seen as complete, as it can support diverse use cases. For smaller, more specialized domains, completeness depends heavily on the specific use cases it supports. This difference implies that DSLs for larger domains are more likely to attract a broader user base, whereas those for narrower domains may find limited but dedicated users.

Maintainability, assessed by evaluating the coupling between individual language statements, is another soft criterion. While intuitive, the assumption that loosely coupled code, where statements are independent and changes in one do not affect others, is easier to maintain is not universally measurable. Nonetheless, DSLs that minimize shared dependencies, such as label references, generally align better with this principle.

The inherent subjectivity in these evaluations underscores the complexity of creating universally applicable metrics for DSL quality while highlighting the importance of defining and contextualizing criteria for consistent analysis.

We designed PQL with the key qualities outlined in Section 2 as the guiding principles. To validate our design, we implemented a language interpreter, which allowed us to detect and correct many design errors and inconsistencies. While this iterative approach proved effective in identifying potential issues, it cannot guarantee complete correctness and consistency of the language. Nevertheless, it has been successful in supporting all of our use cases.

In Section 5, we analyze numerous use cases organized into categories such as event log filtering, projection, calculating key performance indicators (KPIs), aggregations, reordering events, and limiting results. These use cases clearly demonstrate that PQL is capable of efficiently expressing complex event log processing pipelines. Even in the most intricate scenarios, as shown in Section 5.8, PQL queries remain concise — typically involving only a few clauses and terms, and often requiring tens to hundreds of characters to perform the necessary operations. This highlights the expressiveness and simplicity of the language, aligning with the design goals of providing both a powerful and user-friendly tool for event log analysis.

PQL stands out as the only DSL for event logs in Table 1 that encompasses all the identified key qualities. One of the most notable strengths of PQL, compared to other languages, is its comprehensive documentation. While many DSLs lack thorough documentation — often limited to a scientific paper with some examples or an abstract grammar — this manuscript provides a complete specification for

PQL. This includes detailed low-level aspects, error handling, and other essential features. Such comprehensive documentation is crucial for ensuring the consistency and reliability of implementations, especially as the language is adopted and extended. By providing a full specification, PQL sets itself apart as a robust, well-documented solution for event log processing.

We currently consider PQL mature enough to publish the initial draft of its specification. We have already included the PQL interpreter in ProcessM, a fully-featured open-source process mining software [50], to make it available to a broader audience. We are committed to the continuous development, growth, and improvement of PQL. We welcome contributions from the community, including suggestions, fixes, new keywords, and syntax enhancements. Alongside the development of the language, we will also continue to develop and maintain the software tool, which will serve as an essential resource for users of PQL. Through this ongoing effort, we aim to ensure that PQL evolves in line with the needs of the user community.

7. Conclusions and future work

We proposed the Process Query Language (PQL), a domain-specific language (DSL) specifically designed for querying event logs compliant with the IEEE 1849 standard (XES) [4]. PQL is thoroughly documented, with detailed low-level information that meets the requirements for standardization. It has been carefully crafted to adhere to all key qualities of successful, widely adopted DSLs. The design of PQL was empirically validated through a proof-of-concept software implementation,^{1,3} hundreds of unit tests, and numerous use case-based tests, ensuring its practical effectiveness and reliability in real-world applications.

Despite our best efforts, the PQL specification may still require revisions. Some low-level details might be missing, and certain design choices may prove inadequate in specific use cases. Additionally, there may be useful event log processing features yet to be implemented. Therefore, the development of PQL is ongoing. We are publishing this draft specification to highlight the need for standardized tools to manipulate event logs and to encourage the process mining community to build upon PQL. We hope that the community will ultimately develop a fully-featured and perfectly tailored DSL for event logs.

The success of PQL hinges not only on the language's quality and features but also on efforts to gain recognition within the process mining community and inform software vendors about the advantages of a unified language for querying event log databases. Thus far, we have primarily pursued academic recognition by developing both the language and its interpreter within the ProcessM software [50] for process mining. To maximize impact, we have made both the specification and implementation open-source, utilizing established software development technologies. Future research should explore various implementations with diverse code bases and database management systems to uncover practical technical challenges.

Acknowledgment

This work was funded by The National Centre for Research and Development, Poland, under the LIDER X Programme, grant no. LIDER/14/0086/L-10/18/NCBR/2019.

References

- [1] ISO 8879:1986 standard: Information processing — Text and office systems — Standard Generalized Markup Language (SGML). Standard, ISO, Geneva, CH, 1986.
- [2] ISO 9834-8:2014 standard: Information technology — procedures for the operation of object identifier registration authorities — part 8: Generation of universally unique identifiers (uuids) and their use in object identifiers. Standard, International Organization for Standardization, Geneva, CH, 2014.
- [3] ISO 4217:2015 standard: Codes for the representation of currencies. Standard, International Organization for Standardization, Geneva, CH, 2015.
- [4] IEEE 1849-2016 Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams. Standard, IEEE, 2016.
- [5] IEEE 1003.1-2017 standard for information technology—portable operating system interface (POSIX(TM)) base specifications, issue 7. Standard, IEEE, 2017.
- [6] IEEE 754-2019 standard for floating-point arithmetic. Standard, IEEE, 2019.
- [7] ISO 8601-1:2019 standard: Date and time format. Standard, International Organization for Standardization, Geneva, CH, 2019.
- [8] Iso/iec 9075 standard: Information technology – Database languages – SQL. Standard, ISO, Geneva, CH, 2019.
- [9] Álvarez J. M. P., Díaz A. C., Parody L., Quintero A. M. R., and Gómez-López M. T. *Process Instance Query Language and the Process Querying Framework*, pages 85–111. Springer International Publishing, Cham, 2022.
- [10] Alves De Medeiros A., Weijters A., and van der Aalst W. Genetic process mining: an experimental evaluation. *Data Mining and Knowledge Discovery*, 14(2):245–304, 2007.
- [11] Andrews R., Suriadi S., Ouyang C., and Poppe E. Towards event log querying for data quality. In *On the Move to Meaningful Internet Systems. OTM 2018 Conferences*, pages 116–134, Cham, 2018. Springer International Publishing.
- [12] Beheshti S.-M.-R., Benatallah B., Motahari-Nezhad H. R., and Sakr S. A query language for analyzing business processes execution. In *Business Process Management*, pages 281–297, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

-
- [13] Blackwell A. F., Britton C., Cox A., Green T. R. G., Gurr C., Kadoda G., Kutar M. S., Loomes M., Nehaniv C. L., Petre M., Roast C., Roe C., Wong A., and Young R. M. Cognitive dimensions of notations: Design tools for cognitive technology. In *Cognitive Technology: Instruments of Mind*, pages 325–341, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
 - [14] Boubeta-Puig J., Ortiz G., and Medina-Bulo I. MEdit4CEP: A model-driven solution for real-time decision making in SOA 2.0. *Knowledge-Based Systems*, 89:97–112, 2015.
 - [15] Boubeta-Puig J., Ortiz G., and Medina-Bulo I. Model4CEP: Graphical domain-specific modeling languages for cep domains and event patterns. *Expert Systems with Applications*, 42(21):8095–8110, 2015.
 - [16] Carmona J. Projection approaches to process mining using region-based techniques. *Data Min. Knowl. Discov.*, 24(1):218–246, 2012.
 - [17] Cass S. Top programming languages. <https://spectrum.ieee.org/top-programming-languages-2024>, 2024.
 - [18] Costa F., Silva V., de Oliveira D., Ocaña K., Ogasawara E., Dias J., and Matoso M. Capturing and querying workflow runtime provenance with PROV: A practical approach. In *Proceedings of the Joint EDBT/ICDT 2013 Workshops*, EDBT '13, page 282–289, New York, NY, USA, 2013. Association for Computing Machinery.
 - [19] CSS specifications. Standard, W3C, 2021.
 - [20] Dallagassa M. R., dos Santos Garcia C., Scalabrin E. E., Ioshii S. O., and Carvalho D. R. Opportunities and challenges for applying process mining in healthcare: a systematic mapping study. *Journal of Ambient Intelligence and Humanized Computing*, 13(1):165–182, Jan 2022.
 - [21] Dijcks J.-P. Why does it take forever to build ETL processes? <http://blogs.oracle.com/datawarehousing/why-does-it-take-forever-to-build-etl-processes-v2>, 2009. accessed 2024-11-07.
 - [22] Dijkstra E. W. On the cruelty of really teaching computing science, 1988.
 - [23] The decision model and notation specification version 1.3. Standard, Object Management Group, 2021.
 - [24] dos Santos Garcia C., Meincheim A., Faria Junior E. R., Dallagassa M. R., Sato D. M. V., Carvalho D. R., Santos E. A. P., and Scalabrin E. E. Process mining techniques and applications – a systematic mapping study. *Expert Systems with Applications*, 133:260–295, 2019.
 - [25] Durand J., Cho H., Moberg D., and Woo J. XTemp: Event-driven testing and monitoring of business processes: Leveraging XML, XPath and XSLT for a practical event processing. In *Balisage Series on Markup Technologies*, volume 7, Montréal, Canada, 2-5 Aug. 2011.

- [26] Eckel B. and Isakova S. *Atomic Kotlin*. Mindview LLC, 2021.
- [27] El Kamouchi H., Kissi M., and El Beggar O. Low-code/no-code development: A systematic literature review. In *2023 14th International Conference on Intelligent Systems: Theories and Applications (SITA)*, pages 1–8, 2023.
- [28] EsperTech Inc. Esper. <http://espertech.com>. accessed 2024-11-07.
- [29] Esser S. and Fahland D. Multi-dimensional event data in graph databases. *Journal on Data Semantics*, 10(1):109–141, Jun 2021.
- [30] Francis N., Green A., Guagliardo P., Libkin L., Lindaaker T., Marsault V., Plantikow S., Rydberg M., Selmer P., and Taylor A. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 international conference on management of data*, pages 1433–1445, 2018.
- [31] Gonzalez Lopez de Murillas E. *Process mining on databases: extracting event data from real-life data sources*. Phd thesis 1 (research TU/e / graduation TU/e), Mathematics and Computer Science, Feb. 2019. Proefschrift.
- [32] González López de Murillas E., Reijers H. A., and van der Aalst W. M. P. Everything you always wanted to know about your process, but did not know how to ask. In *Business Process Management Workshops*, pages 296–309, Cham, 2017. Springer International Publishing.
- [33] Green T., Blandford A., Church L., Roast C., and Clarke S. Cognitive dimensions: Achievements, new directions, and open questions. *Journal of Visual Languages & Computing*, 17(4):328–365, 2006. Ten Years of Cognitive Dimensions.
- [34] Haan K. and Watts R. Top website statistics for 2024. <https://www.forbes.com/advisor/business/software/website-statistics/>, 2024. accessed 2024-11-07.
- [35] Hazel P. Perl compatible regular expressions. Technical report, 2021.
- [36] HTML living standard. Standard, WHATWG, 2024.
- [37] Jones C. Programming languages table release 8.2. <http://www.cs.bsu.edu/homepages/dmz/cs697/langtbl.htm>, 1996. accessed 2024-11-07.
- [38] Jungmann L., Matějček D., Kornilov D., Bratt E., King G., Filho I. J., Grassel J., Dazey W., Shannon B., Cortez R., Liu S., Tian W., and B Y. K. Jakarta persistence API 3.0, 2020.
- [39] Kobeissi M., Assy N., Gaaloul W., Defude B., Benatallah B., and Haidar B. Natural language querying of process execution data. *Information Systems*, 116:102227, 2023.

-
- [40] La Rosa M., Reijers H. A., van der Aalst W. M., Dijkman R. M., Mendling J., Dumas M., and García-Bañuelos L. Apromore: An advanced process model repository. *Expert Systems with Applications*, 38(6):7029–7040, 2011.
 - [41] Liu D., Pedrinaci C., and Domingue J. Semantic enabled complex event language for business process monitoring. In *Proceedings of the 4th International Workshop on Semantic Business Process Management*, SBPM '09, page 31–34, New York, NY, USA, 2009. Association for Computing Machinery.
 - [42] Marin-Castro H. M. and Tello-Leal E. An end-to-end approach and tool for bpmn process discovery. *Expert Systems with Applications*, 174:114662, 2021.
 - [43] Marin-Castro H. M. and Tello-Leal E. Event log preprocessing for process mining: A review. *Applied Sciences*, 11(22), 2021.
 - [44] Mernik M., Heering J., and Sloane A. M. When and how to develop domain-specific languages. *ACM Comput. Surv.*, 37(4):316–344, Dec. 2005.
 - [45] Microsoft. Streaminsight. [http://docs.microsoft.com/en-us/previous-versions/sql/streaminsight/ee362541\(v=sql.111\)](http://docs.microsoft.com/en-us/previous-versions/sql/streaminsight/ee362541(v=sql.111)), 2016. accessed 2024-11-07.
 - [46] Momotko M. and Subieta K. Process query language: A way to make workflow processes more flexible. In *Advances in Databases and Information Systems*, pages 306–321, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
 - [47] Noghabi S. A., Paramasivam K., Pan Y., Ramesh N., Bringhurst J., Gupta I., and Campbell R. H. Samza: stateful scalable stream processing at linkedin. *Proc. VLDB Endow.*, 10(12):1634–1645, Aug. 2017.
 - [48] Nordmann A., Hochgeschwender N., and Wrede S. A survey on domain-specific languages in robotics. In *Simulation, Modeling, and Programming for Autonomous Robots*, pages 195–206, Cham, 2014. Springer International Publishing.
 - [49] Parr T. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2nd edition, 2013.
 - [50] Pawlak T. P. and Potoniec J. Processm: Intelligent process mining software. *SoftwareX*, 30:102079, 2025.
 - [51] Perez-Alvarez J. M., Gomez-Lopez M. T., Parody L., and Gasca R. M. Process instance query language to include process performance indicators in DMN. In *2016 IEEE 20th International Enterprise Distributed Object Computing Workshop (EDOCW)*, pages 1–8, 2016.
 - [52] Polyvyanyy A., ter Hofstede A. H., La Rosa M., Ouyang C., and Pika A. Process query language: Design, implementation, and evaluation. *Information Systems*, 122:102337, 2024.
 - [53] Poppe O., Giessel S., Rundensteiner E. A., and Bry F. *The HIT Model: Workflow-Aware Event Stream Monitoring*, pages 26–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.

- [54] Prechelt L. An empirical comparison of seven programming languages. *Computer*, 33(10):23–29, 2000.
- [55] Raja A. and Lakshmanan D. Domain specific languages. *International Journal of Computer Applications*, 1(21):99–105, February 2010.
- [56] JavaScript object notation (JSON) data interchange format. Standard, IETF, Dec. 2017.
- [57] Schönig H.-J. *Mastering PostgreSQL 13*. Packt Publishing, 4th edition, 2020.
- [58] solid IT gmbh. DB Engines: Knowledge base of relational and NoSQL database management systems, 7 Nov. 2024.
- [59] SPARQL 1.1 overview: W3C Recommendation. Standard, W3C, 2013.
- [60] Scalable vector graphics (SVG) 2. Standard, W3C, Oct. 2018.
- [61] Tang Y., Cui W., and Su J. A query language for workflow logs. *ACM Trans. Manage. Inf. Syst.*, 13(2), Dec. 2021.
- [62] Teinemaa I., Dumas M., Leontjeva A., and Maggi F. M. Temporal stability in predictive process monitoring. *Data Min. Knowl. Discov.*, 32(5):1306–1338, 2018.
- [63] ter Hofstede A. H. M., Ouyang C., La Rosa M., Song L., Wang J., and Polyvyanyy A. APQL: A process-model query language. In *Asia Pacific Business Process Management*, pages 23–38, Cham, 2013. Springer International Publishing.
- [64] The Apache Software Foundation. Cassandra. <http://cassandra.apache.org>, 2021. accessed 2024-11-07.
- [65] van der Aalst W. M. P. *Process Mining: Data Science in Action*. Springer, Heidelberg, 2nd edition, 2016.
- [66] van der Aalst W. M. P., Bolt A., and van Zelst S. J. RapidProM: mine your processes and not just your data. *CoRR*, abs/1703.03740, 2017.
- [67] van Deursen A., Klint P., and Visser J. Domain-specific languages: An annotated bibliography. *SIGPLAN Not.*, 35(6):26–36, June 2000.
- [68] van Dongen B. Real-life event logs - Hospital log, 3 2011.
- [69] van Dongen B. F., de Medeiros A. K. A., Verbeek H. M. W., Weijters A. J. M. M., and van der Aalst W. M. P. The ProM framework: A new era in process mining tool support. In *Applications and Theory of Petri Nets 2005*, pages 444–454, Berlin, 2005. Springer.

- [70] van Dongen B. F. and Shabani S. Relational XES: Data management for process mining. In *Proceedings of the CAiSE 2015 Forum at the 27th International Conference on Advanced Information Systems Engineering co-located with 27th International Conference on Advanced Information Systems Engineering (CAiSE 2015), Stockholm, Sweden, June 10th, 2015*, volume 1367 of *CEUR Workshop Proceedings*, pages 169–176. CEUR-WS.org, 2015.
- [71] van Steen M. and Tanenbaum A. *Distributed Systems*. distributed-systems.net, 3 edition, 2017.
- [72] Visser E. WebDSL: A case study in domain-specific language engineering. In *Generative and Transformational Techniques in Software Engineering II, International Summer School, GTTSE 2007*, volume 5235 of *Lecture Notes in Computer Science*, pages 291–373, Braga, Portugal, 2007. Springer.
- [73] Vogelgesang T., Ambrosy J., Becher D., Seilbeck R., Geyer-Klingenberg J., and Klenk M. *Celonis PQL: A Query Language for Process Mining*, pages 377–408. Springer International Publishing, Cham, 2022.
- [74] Wen L., van der Aalst W. M. P., Wang J., and Sun J. Mining process models with non-free-choice constructs. *Data Min. Knowl. Discov.*, 15(2):145–180, 2007.
- [75] Extensible markup language (XML) 1.0 (fifth edition). Standard, W3C, Feb. 2008.
- [76] XML testing and event-driven monitoring of processes (XTemp) version 1.0. Standard, OASIS, 2011.

Received 21.11.2024, Accepted 21.05.2025

Appendix A. PQL specification

A.1. Introduction

Process Query Language (PQL) is a language for querying event logs for efficient retrieval of process-related information, such as process variants and key performance indicators. A PQL query forms a data source for business process analytics tools, such as the tools for process discovery, conformance checking, root cause analysis, and process enhancement. PQL enables a user to specify a view on a collection of event logs available to him or her. PQL is inspired by the ISO/IEC 9075:2016 standard for Information technology — Database languages — SQL [8], however, there are clearly visible differences. This document summarizes the data model, syntax, and features of PQL.

A.2. Case sensitivity

All keywords, identifiers, and comparisons with values in PQL are case-sensitive.

A.3. Data model

PQL query works on a data structure compatible with the IEEE 1849-2016 Standard for eXtensible Event Stream for Achieving Interoperability in Event Logs and Event Streams [4] (XES). The data source is a list of XES-conformant logs. Each log is a hierarchical structure with three levels. The root of this hierarchy is the **log** component. It is associated with a list of **trace** components. In turn, a trace consists of a list of **event** components. An individual log usually corresponds to an individual business process, a trace corresponds to a business case in this process, and an event corresponds to an event in this business case. Logs, traces, and events consist of attributes. This structure is visualized using parent-child relations:

```
logs
└─traces
   └─events
```

Every **event** at the **events** level is a child of exactly one **trace** at the **traces** level, and every **trace** is a child of exactly one **log** at the **logs** level.

The result of a PQL query is a projection of this structure that maintains the same three-level hierarchy of relations.

A.3.1. Scopes

A scope is a view on the data model limited to its single part. PQL defines the following scopes:

- **log** - refers to the **logs** level,
- **trace** - refers to the **traces** level,
- **event** - refers to the **events** level.

PQL defines **shorthand scope names** **l**, **t**, and **e** for **log**, **trace**, and **event** scopes, respectively.

Each time a scope is expected in a PQL query but not specified explicitly, the **event** scope is used.

A.3.2. Scope hoisting

PQL defines a hoisting prefix $\sim$ for a scope. It moves the referenced expression from its scope to its parent scope. It is allowed to duplicate $\sim$ prefix to hoist the scope further. E.g., $\sim\text{event}$ moves the scope of an expression from the **event** scope to

the `trace` scope, and `^^event` moves it to the `log` scope. However, it is forbidden to hoist a scope beyond the `log` scope, hence `^^^event` and `^^trace` are incorrect.

Since the parent-child relation is one-to-many, the hoisted expression effectively holds the *list* of its values on its original scope.

Scope hoisting is useful in filtering using the `where` clause, where it allows for filtering entries at certain scope using the expressions made of the attributes of its children scopes. The `where` clause is satisfied if it holds for any value in the list of the values of the hoisted expression.

Scope hoisting is also useful in grouping using the `group by` clause, where it allows for grouping of traces into process variants using the attributes of the events.

Scope hoisting is not supported in the `select` and the `order by` clauses, except as an argument to an aggregation function.

A.3.3. Data types

PQL distinguishes the data types below:

- `uuid` - an universally unique identifier,
- `string` - an UTF-8-encoded text,
- `number` - a double-precision floating point number compliant with the IEEE 754-2019 standard [6],
- `datetime` - an UTC timestamp with millisecond precision compliant with the ISO 8601-1:2019 standard [7],
- `boolean` - a Boolean-algebra `true` or `false`,
- `any` - any of the above-mentioned.

Every data type includes a special `null` value that represents a lack of the actual value. All comparisons to `null` yield `false`, except for a special `is` operator.

PQL does not support type casts.

A.3.4. Literals

PQL supports the use of literals in queries. The representation of the literal depends on its type:

- A `uuid` is a universally unique identifier compliant with the ISO/IEC 9834-8:2014 standard [2], 32 hexadecimal (base-16) digits, displayed in five groups separated by hyphens, in the form of 8-4-4-4-12,
- A `string` literal is a `'single-quoted'` or `"double-quoted"` string; backslash `\` can be used as an escape character, see below for the details,
- A `number` literal is a valid IEEE 754-2019 [6] string representation of a number, such as e.g., the decimal point number `3.14` and the scientific notation number `1.23E45`,
- A `datetime` literal consists of a prefix `D` and a date and time with timezone in the format compliant with the ISO 8601-1:2019 standard [7], where time and

- timezone are optional parts, e.g., D2020-03-13T16:45:50.333, D2020-03-13T16:45+02:00, D20200313, D20200313164550.333,
- A `boolean` literal is either `true` or `false`.

The literals are scopeless by default, i.e., they do not change the scope of the expression. However, it is supported to specify explicitly the scope of the literal using a scope prefix, e.g., `l:'log-scoped string'`, `t:D2020-03-13T16:45+02.00`, and `e:3.14` are valid literals with the scopes specified. If an expression reduces to scopeless literals only, i.e., consists of no attributes, and no literal has a scope associated with, then the default scope of `event` applies.

Escape sequences in string literals The `string` literals in PQL may contain escape sequences. An escape sequence consists of the backslash character `\` and the sequence of one or more characters. The table below shows the complete list of available escape sequences.

Escape sequence	Meaning
<code>\b</code>	Backspace
<code>\n</code>	New line
<code>\t</code>	Horizontal tab
<code>\f</code>	Form feed
<code>\r</code>	Carriage return
<code>\o, \oo, \ooo</code>	Octal byte value, where <code>o</code> is a number from the range from 0 to 7
<code>\uxxxx</code>	16-bit Unicode character, where <code>x</code> is a hexadecimal number from the range from 0 to F

Any other character following the backslash `\` is read literally. To include a backslash character, write two backslashes `\\`; to include a single-quote in the single-quoted string use `\'`, and to include a double-quote in the double-quoted string use `\"`.

It is the user's responsibility that the byte sequences created using the octal and Unicode escape sequences are valid characters in the UTF-8 encoding. The `\uxxxx` escape sequence can be used to specify UTF-16 surrogate pairs to compose characters with code points larger than `\uFFFF`.

A.3.5. Attributes

The set of attributes available on every scope dynamically adapts to data: every attribute supplied by the data source is available for use. It is even possible that every log, trace, and event has a different set of attributes associated with it.

The standard attributes PQL defines **standard attributes** available at all times. A name of a standard attribute is a colon-separated list of the scope, the extension prefix, and the name of the attribute as defined in the IEEE 1849-2016: IEEE

Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams [4].

A complete list of standard attributes is given below:

- **log:concept:name** (type: **string**) - Stores a generally understood name for the log. Usually the name of the process having been executed.
- **log:identity:id** (type: **uuid**) - Unique identifier (UUID) for the log.
- **log:lifecycle:model** (type: **string**) - This attribute refers to the lifecycle transactional model used for all events in the log. If this attribute has a value of “standard”, the standard lifecycle transactional model is assumed. If it is a value of “bpaf”, the Business Process Analytics Format (BPAF) lifecycle transactional model is assumed. See the IEEE 1849-2016 [4] for details.
- **log:xes:version** (type: **string**) - The version of the XES standard the log conforms to (e.g., 1.0).
- **log:xes:features** (type: **string**) - A whitespace-separated list of optional XES features (e.g., **nested-attributes**). If no optional features are used, this attribute shall have an empty value.
- **trace:concept:name** (type: **string**) - Stores a generally understood name for the trace. Usually the case ID.
- **trace:cost:currency** (type: **string**) - The currency (using the ISO 4217:2015 standard [3]) of all costs of this trace.
- **trace:cost:total** (type: **number**) - Total cost incurred for a trace. The value represents the sum of all the cost amounts within the element.
- **trace:identity:id** (type: **uuid**) - Unique identifier (UUID) for the trace.
- **trace:classifier:<name>** (type: **string**) - Refers the classifier for the **trace** scope using its log-specific **<name>**. The **<name>** must follow the regular expression `^[a-zA-Z\u0080-\u00FF_0-9]+$`. For the classifiers not compliant with this naming, the use of the square bracket syntax is required (see below). A classifier is a list of attributes, whose values give identity to a trace. See IEEE 1849-2016 [4] for the details on classifiers. Note that different logs returned by the same query may define different classifiers, and so the definition of a specific classifier visible to traces from each log may be different. The use of this attribute is allowed only in the **select**, the **group by**, and the **order by** clauses, where it evaluates to a collection of attributes. It is strictly prohibited to use this attribute in other clauses.
- **event:concept:name** (type: **string**) - Stores a generally understood name for an event. Usually the name of the executed activity represented by the event.
- **event:concept:instance** (type: **string**) - This represents an identifier of the activity instance whose execution has generated the event. This way, multiple instances (occurrences) of the same activity can be told apart.
- **event:cost:currency** (type: **string**) - The currency (using the ISO 4217:2008 standard) of all costs of this event.
- **event:cost:total** (type: **number**) - Total cost incurred for an event. The value represents the sum of all the cost amounts within the element.
- **event:identity:id** (type: **uuid**) - Unique identifier (UUID) for the event.
- **event:lifecycle:transition** (type: **string**) - The transition attribute is de-

defined for events, and specifies the lifecycle transition of each event. The transitions following the *standard* model should use this attribute. See IEEE 1849-2016 [4] for the details.

- **event:lifecycle:state** (type: **string**) - The state attribute is defined for events and specifies the lifecycle state of each event. The transitions following the *BPAF* model should use this attribute. See IEEE 1849-2016 [4] for the details.
- **event:org:resource** (type: **string**) - The name, or identifier, of the resource that triggered the event.
- **event:org:role** (type: **string**) - The role of the resource that triggered the event, within the organizational structure.
- **event:org:group** (type: **string**) - The group within the organizational structure, of which the resource that triggered the event is a member.
- **event:time:timestamp** (type: **datetime**) - The UTC time at which the event occurred.
- **event:classifier:<name>** (type: **string**) - Refers the classifier for the **event** scope using its log-specific **<name>**. The **<name>** must follow the regular expression `^[a-zA-Z\u0080-\u00FF_0-9]+$`. For the classifiers not compliant with this naming, the use of the square bracket syntax is required (see below). A classifier is a list of attributes, whose values give identity to an event. See IEEE 1849-2016 [4] for the details on classifiers. Note that different logs returned by the same query may define different classifiers, and so the definition of a specific classifier visible to events from each log may be different. The use of this attribute is allowed only in the **select**, the **group by**, and the **order by** clauses, where it evaluates to a collection of attributes. It is forbidden to use this attribute in the other clauses.

The attributes provided by the data source are translated to these standard names using XES extensions as defined in the IEEE 1849-2016 standard [4], even if the name of the attribute differs in the data source. The translation works in three steps:

- For each log in the data source separately read the log-specified prefixes for all standard XES extensions explicitly attached to this log. The standard XES extension is recognized by URI, e.g., <http://xes-standard.org/concept.xesext> refers to the **concept** extension. Note that different logs may define different prefixes for the same standard attributes.
- For each log in the data source separately, for each unattached standard XES extension attach it with its standard prefix as defined in the IEEE 1849-2016 standard [4],
- For each log in the data source separately, for each attribute in it change its prefix to the standard prefix as defined in the IEEE 1849-2016 standard [4].

Only the attributes from the XES extensions attached to the log are translated this way. The standard attributes that do not translate to any attribute in the log, are left **null**. Note that the attributes remain available under their original names when using the square bracket syntax (see below).

Note that the attributes `trace:classifier:<name>` and `event:classifier:<name>` are extracted from the collection of XES classifiers provided by the data source with the log and their names do not follow that translation rule, as the IEEE 1849-2016 standard [4] does not define classifier prefixes. However, the list of the attributes corresponding to the classifier follows translation accordingly.

It is allowed to use **shorthand names** for the standard attributes by omitting the standard prefix of the defining XES extension and the following colon. E.g., `event:concept:name` and `event:name` refer to the same attribute. Given the shorthand rules for the scopes defined above, `name`, `e:name`, and `e:concept:name` refer to `event:concept:name` too. This does not apply to `trace:classifier:<name>` and `event:classifier:<name>`, whose prefixes must not be omitted, but can be shortened to `c`, resulting in `trace:c:<name>` and `event:c:<name>`.

The bracket syntax for non-standard attributes All other attributes are considered non-standard. The non-standard attributes are available using the square bracket syntax:

- `[log:<attribute>]` - retrieves the `<attribute>` of a log,
- `[trace:<attribute>]` - retrieves the `<attribute>` of a trace,
- `[event:<attribute>]` - retrieves the `<attribute>` of an event,
- `[trace:classifier:<name>]` - retrieves the trace classifier with `<name>`; see above for the details on classifiers,
- `[event:classifier:<name>]` - retrieves the event classifier with `<name>`; see above for the details on classifiers.

The square bracket syntax uses the untranslated fully-qualified names of the attributes as provided by the data source.

The value of an attribute non-existent in a particular component referred to using the bracket syntax evaluates to `null` for this component. This behavior simplifies operations on components with different sets of attributes. Note that the IEEE 1849-2016 standard [4] allows for different sets of attributes for every event within the same trace, and for every trace within the same log.

A.4. Identifiers

An identifier refers to the formal object representing a data item. A valid identifier is any of the following:

- Attribute name.

Note that the term *identifier*, although equivalent to the *attribute name*, is distinguished as a more general formal object reserved for future use.

A.5. Functions

PQL supports aggregation and scalar functions of attributes, literals, and an expression thereof. The functions in PQL do not cause side effects.

A.5.1. Aggregation functions

PQL supports aggregation functions. For a query with the `group by` clause, an aggregation function yields a single value for each group on each scope. For a query without the `group by` clause, use of the aggregation function triggers the implicit group by clause. Aggregation functions can be used in the `select` and `order by` clauses. They cannot appear in the `where`, the `group by`, the `limit`, and the `offset` clauses.

The complete list of aggregation functions is as follows:

- `min(any)` -> `any` - Returns the minimum non-null value in the set of values of the given attribute for a group, or `null` if such value does not exist. See comparison operators for details.
- `max(any)` -> `any` - Returns the maximum non-null value in the set of values of the given attribute for a group, or `null` if such value does not exist. See comparison operators for details.
- `avg(number)` -> `number` - Returns the average of non-null values of the given attribute for a group.
- `count(any)` -> `number` - Returns the count of non-null values of the given attribute for a group.
- `sum(number)` -> `number` - Returns the sum of non-null values of the given attribute for a group.

The aggregation function can only take an identifier as an argument.

A.5.2. Scalar functions

A scalar function takes zero or more scalar arguments and yields a new value.

The complete list of scalar functions is as follows:

- `date(datetime)` -> `datetime` - Returns the date part of a datetime (hours, minutes, seconds and milliseconds are zeroed).
- `time(datetime)` -> `datetime` - Returns the time part of a datetime (year, month, day are zeroed).
- `year(datetime)` -> `number` - Returns the year from a datetime.
- `month(datetime)` -> `number` - Returns the month (1-12) from a datetime.
- `day(datetime)` -> `number` - Returns the day of month (1-31) from a datetime.
- `hour(datetime)` -> `number` - Returns the hour value (0-23) from a datetime.
- `minute(datetime)` -> `number` - Returns the minute value (0-59) from a datetime.

- `second(datetime)` -> `number` - Returns the second value (0-59) from a datetime.
- `millisecond(datetime)` -> `number` - Returns the millisecond value (0-999) from a datetime.
- `quarter(datetime)` -> `number` - Returns the quarter (1-4) from a datetime.
- `dayofweek(datetime)` -> `number` - Returns the day of week from a datetime. 1 for Sunday, 2 for Monday, 3 for Tuesday etc.
- `now()` -> `datetime` - Returns the datetime of invoking of the query; successive calls to this function in the same query are guaranteed to return the same value.
- `upper(string)` -> `string` - Converts the given string converted to uppercase.
- `lower(string)` -> `string` - Converts the given string converted to lowercase.
- `round(number)` -> `number` - Rounds the given number to the nearest integer value; rounds half away from zero.

A.6. Expressions

A.6.1. Arithmetic expressions

PQL defines arithmetic expressions involving addition `+`, subtraction `-`, multiplication `*`, and division `/` operators. It allows for overwriting standard precedence rules using the round brackets `()`. An arithmetic expression may use a valid identifier, a function call, a scalar value, and any combination thereof.

E.g., `e:cost:total`, `e:cost:total * [e:currency-rate:EURtoUSD]`, `avg(e:cost:total)`, and `(e:cost:total + 10) * [e:currency-rate:EURtoUSD]` are valid arithmetic expressions.

A.6.2. Logical expressions

PQL defines logic expressions involving `and`, `or`, and `not` operators. It allows for overwriting standard precedence rules using the round brackets `()`. A logical expression may use `boolean` attributes, literals `true` and `false`, comparisons of arithmetic expressions, and a combination thereof.

E.g., `e:org:resource = 'scott'`, `e:org:resource='scott' or e:org:group='helpdesk'`, and `(e:org:resource='scott' or e:org:group='helpdesk') and dayofweek(e:time:timestamp)=1` are valid logic expressions.

A.6.3. Properties of an expression

Every expression is characterized by the scope. The scope of the expression is calculated in three steps:

- Collect the set of the scopes with the optional hoisting prefix of all attributes,

literals, and functions in this expression,

- Hoist the scopes in this set using the associated hoisting prefix wherever set,
- Select the lowest scope from this set or the **event** scope if empty.

A.7. Operators

This section summarizes operators available in PQL.

A.7.1. Arithmetic operators

- `number * number -> number` - multiplication of two numbers.
- `number / number -> number` - multiplication of two numbers.
- `number + number -> number` - addition of two numbers.
- `number - number -> number` - subtraction of two numbers.

All arithmetic operators yield **null** if any of their operands is **null**.

A.7.2. Text operators

- `string + string -> string` - concatenation of two strings.
- `string like string -> boolean` - evaluates to **true** if and only if the first string matches from the beginning to the end the pattern specified by the second string, **false** otherwise. The pattern consists of any characters interleaved with zero or more placeholder characters. The following placeholder characters are defined:
 - `_` - matches exactly one any character,
 - `%` - matches zero or more characters. To match a literal `_` or `%` without matching other characters, the respective character in the pattern must be preceded by the escape character `\`. To match the escape character itself, write two escape characters.
- `string matches string -> boolean` - evaluates to **true** if and only if the first string matches the regular expression specified by the second string, **false** otherwise. The match may occur anywhere within the string unless the regular expression is explicitly anchored to the beginning or the end of the string. The regular expression must conform to the IEEE 1003-1:2017 Standard for Information Technology–Portable Operating System Interface (POSIX(R)) Base Specifications [5].

A.7.3. Temporal operators

- `datetime - datetime -> number` - subtraction of two **datetimes**, the resulting value is the number of days between them with fractional part representing

the fraction of the day.

All temporal operators yield **null** if any of their operands is **null**.

A.7.4. Logic operators

- **boolean and boolean** -> **boolean** - Boolean conjunction.
- **boolean or boolean** -> **boolean** - Boolean disjunction.
- **not boolean** -> **boolean** - Boolean negation.

All logic operators yield **null** if any of their operands is **null**.

A.7.5. Comparison operators

- **any = any** -> **boolean** - Evaluates to **true** if and only if both values have the same type and value, **false** otherwise.
- **any != any** -> **boolean** - Evaluates to **false** if and only if both values have the same type and value, **true** otherwise.
- **any < any** -> **boolean** - Evaluates to **true** if and only if both arguments have the same type and the first argument is smaller than the second, **false** otherwise.
- **any <= any** -> **boolean** - Evaluates to **true** if and only if both arguments have the same type and the first argument is smaller than or equal to the second, **false** otherwise.
- **any > any** -> **boolean** - Evaluates to **true** if and only if both arguments have the same type and the first argument is larger than the second, **false** otherwise.
- **any >= any** -> **boolean** - Evaluates to **true** if and only if both arguments have the same type and the first argument is larger than or equal to the second, **false** otherwise.

For these operators, **false** is considered smaller than **true**, numbers are compared using the IEEE 754-2019 [6] rules, **datetimes** are compared with earlier being smaller, strings are compared alphabetically, with case-sensitivity. Comparison to **null** yields **false**.

A.7.6. Type-independent operators

- **any is null** -> **boolean** - Evaluates to **true** if and only if **any** is **null**, **false** otherwise.
- **any is not null** -> **boolean** - Evaluates to **true** if and only if **any** is not **null**, **false** otherwise.
- **any in (any, any, ..., any)** -> **boolean** - Evaluates to **true** if and only if the first **any** equals any of the remaining values, **false** otherwise.

- `any not in (any, any, ..., any)` -> `boolean` - Evaluates to `false` if and only if the first `any` equals any of the remaining values, `true` otherwise.

A.7.7. Operator precedence

This table orders the operators in the descending precedence:

Precedence	Operator	Associativity
1	<code>*</code> , <code>/</code>	left
2	<code>+</code> , <code>-</code>	left
3	<code>in</code> , <code>not in</code> , <code>like</code> , <code>matches</code>	none
4	<code>=</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	none
5	<code>is null</code> , <code>is not null</code>	none
6	<code>not</code>	right
7	<code>and</code>	left
8	<code>or</code>	left

A.8. Syntax

A PQL query is a list of the below clauses in the order specified below. All clauses are optional. An empty query is a valid query and returns all data in the data source.

- `select` or `delete`
- `where`
- `group by`
- `order by`
- `limit`
- `offset`

A.8.1. The select clause

The select clause specifies the attributes to fetch. The select clause takes one of the forms below:

```
select *
select <scope1>:*[, <scope2>:*[, <scope3>:*[
select <expression1>[, <expression2>[, ...[, <expressionN>]]]
```

Where the first form selects all available attributes from the data model. This is equivalent to not specifying the `select` clause at all. However, for efficient evaluation of complex queries on large data sources, it is recommended to specify attributes explicitly using the third form.

The second form selects all attributes on the given scopes. The third form enables us to select specific expressions defined using attributes, literals, and functions. The user is free to combine the second and the third form by separating the `<scope>:*`-based selectors from the `<expression>`-based selectors using commas.

The expressions in the `select` clause are evaluated and retrieved as attributes associated with the components. The type of the attribute corresponds to the type of the expression. The implementations are free to assign custom names to the attributes created from the expressions.

The IEEE 1849-2016 standard [4] forbids duplicate attribute names in the same component, except for the list attribute. PQL follows this restriction and each time a query selects the same attribute twice, e.g., by a direct reference and using a classifier, only the first reference is retrieved.

E.g., the below query selects the `concept:name` attribute for the `log` scope, the `concept:name` and `cost:currency` attributes for the `trace` scope, and the `concept:name` and `cost:total` attributes for the `event` scope.

```
select l:name, t:name, t:currency, e:name, e:total
```

The below query selects the `concept:name` attribute for the `trace` scope, and all attributes for the `event` scope.

```
select t:name, e: *
```

The below query selects the attributes defined in the ‘businesscase’ classifier for the `trace` scope and defined in the `activity_resource` classifier for the `event` scope.

```
select t:classifier:businesscase, e:classifier:activity_resource
```

The below query selects the minimum, the average, and the maximum of the total cost for all traces in the data source.

```
select min(t:total), avg(t:total), max(t:total)
```

A.8.2. The delete clause

The `delete` clause causes the removal of the items returned by the query and their descendants. The deletion of a log also deletes all traces and events in this log; the deletion of a trace also deletes all events in this traces. The deletion of an event deletes only this event. The `delete` clause takes one of the forms:

```
delete
delete <scope>
```

The first form is an abbreviation for `delete event`, i.e., `event` is the default scope.

The `delete` clause deletes all returned items, hence the query:

```
delete log
```

deletes all event logs, traces, and events.

For the deletion of all traces and their corresponding events in all event logs, without deleting the logs themselves, use:

delete trace

For the deletion of all events in all traces and all event logs, without deleting the logs and traces themselves, use:

delete event

To filter concrete items for deletion, combine the **delete** clause with the **where** clause. The **delete** clause must not be combined with the **select**, **group by** clauses, and the aggregate functions.

A.8.3. The where clause

The **where** clause *filters* components to be fetched by the PQL query. It takes the form of

where <logical_expression>

Where <logical_expression> refers to an arbitrary logical expression.

E.g., the below query fetches the logs with traces with events at weekends. The logs and traces without weekend events are filtered out. The events in workdays are filtered out.

where **dayofweek**(e:timestamp) **in** (1, 7)

In contrast, the below query fetches the logs with traces with events at weekends, however, it keeps events in other days thanks to scope hoisting. The logs and traces without events at weekends are filtered out.

where **dayofweek**(^e:timestamp) **in** (1, 7)

The next query fetches the logs with traces with events at weekends. The logs without weekend events are filtered out, however, the traces without weekend events but in the logs containing the weekend events are returned.

where **dayofweek**(^^e:timestamp) **in** (1, 7)

The below query selects the logs with traces having the currency of their total cost not reported as the currency of their children's events. All events are kept.

where **not**(t:currency = ^e:currency)

In contrast, the below query filters out the events having the same currency as their corresponding traces.

where t:currency != e:currency

The below query fetches the logs with traces having the currency of their total cost not reported as the currency of their children events and the total cost of the trace is null.

where **not**(t:currency = ^e:currency) **and** t:total **is** null

A.8.4. The group by clause

The **group by** clause clusters the components into groups having the same values of the given attributes. It is possible to specify one or more attributes using the below syntax. If the attribute is a classifier, i.e., **trace:classifier:<name>** or **event:classifier:<name>**, then this attribute expands to a list of actual attributes.

```
group by <attribute1>[, <attribute2>[,...[, <attribute3>]]]
```

The scope of the attribute corresponds to the scope of grouping. Scope hoisting is supported and allows for grouping of the parent-scope components using the list of the values of the child-scope attributes. E.g., the hoisted attribute

^event:concept:name allows for grouping of traces into process variants based on the list of the values of the **concept:name** attribute of the underlying events. As a result, each variant corresponds to the group of traces having the same sequence of events.

The **group by** clause with a scope *S* restricts the attributes available in the **select** and **order by** clauses on scope *S* and children scopes to the attributes specified in this clause. All other attributes may be used as arguments for aggregation functions.

Note that the query without the **select** clause fetches all *available* attributes rather than all attributes. For the queries with the **group by** clause this means that for the grouped scope and the lower scopes only the attributes enumerated in the **group by** clause are fetched.

E.g., the below query selects all logs in the data source with all their attributes, groups the traces into variants using the classifier **event:classifier:activity**, and for each trace variant selects all events with the attributes listed in the classifier definition.

```
group by ^e:classifier:activity
```

The below query selects **trace:concept:name** for each trace, and the sum of total costs for each group of events having the same **event:concept:name** within each trace individually.

```
select t:name, e:name, sum(e:total)
group by e:name
```

The below query seeks for the variants of the traces with the same sequence of events, comparing the events using the **event:concept:name** attribute. The events in the resulting trace variants contain the **event:concept:name** attribute and the sum of the total costs incurred by all events with the same position within the variant.

```
select e:name, sum(e:total)
group by ^e: name, e: name
```

The below query for each group of logs with the same sequence of events among all traces, comparing events using the **event:concept:name** attribute, selects the aggregated log. In the resulting log, the events contain the **event:concept:name** attribute and the sum of the total costs incurred by all events with the same position within this group of logs.

```
select e:name, sum(e:total)
group by ^^e:name, e:name
```

Implicit group by The use of the aggregation function in the `select` or `order by` clause without the use of any attribute of the same scope as this aggregation function in the `group by` clause (or omitting the `group by` clause) implies an implicit `group by` clause for this scope. The implicit `group by` groups all components on this scope into a single aggregation component carrying the aggregated data for all matching components. The query with the implicit `group by` clause on scope *S* must aggregate all attributes on scope *S* referenced in the `select` and `order by` clauses. It is strictly forbidden to use an unaggregated attribute together with the implicit `group by` clause.

E.g., the below query for each log and for each trace yields a single event holding the average total cost and the boundaries of the time window of all events in the corresponding trace.

```
select avg(e:total), min(e:timestamp), max(e:timestamp)
```

The below query yields a single log holding the average cost incurred by and the boundaries of the time window of all events belonging to all logs in the data source.

```
select avg(^^e:total), min(^^e:timestamp), max(^^e:timestamp)
```

A.8.5. The order by clause

The `order by` clause specifies the sorting order of the results. Its syntax is as follows:

```
order by <expression1> [<direction>][, <expression2> [<direction>][,...]]
```

The `<expression*>` placeholders refer to arithmetic expressions, whose values are ordering keys. The `trace:classifier:*` and `event:classifier:*` attributes expand to the list of underlying attributes. The `<direction>` placeholders refer to the ordering direction, either `asc` or `desc` for ascending or descending direction, respectively. When omitted, `asc` is assumed.

The ordering is the same as imposed by the comparison operators, except that the `null` values are considered greater than all other values.

By omitting the `order by` clause, the components are returned in the same order as provided by the data source. E.g., the order of the traces and events in the XES file.

The below query orders the events within a trace ascendingly by their timestamps.

```
order by e:timestamp
```

The below query orders the traces descendingly by their total costs and the events within each trace ascendingly by their timestamps.

```
order by t:total desc, e:timestamp
```


A.8.6. The limit clause

The limit clause imposes the limit on the number of the returned components. It takes the form of:

```
limit <scope>:<number>[, <scope>:<number>[, <scope>:<number>]]
```

where the `<scope>` placeholder refers to the scope on which to impose the limit given by the corresponding `<number>` placeholder.

E.g., the below query returns at most five logs, at most ten traces per log, and at most twenty events per trace.

```
limit 1:5, t:10, e:20
```

A.8.7. The offset clause

The offset clause skips the given number of the beginning entries. It has the below form:

```
offset <scope>:<number>[, <scope>:<number>[, <scope>:<number>]]
```

Where the `<scope>` placeholder refers to the scope on which to impose the limit given by the corresponding `<number>` placeholder.

E.g., the below query returns all but the first five logs, all but the first ten traces per log, and all but twenty events per trace.

```
offset 1:5, t:10, e:20
```

The below query combines `limit` and `offset` clauses to skip the first ten logs and return at most five logs, for each log skip the first twenty traces and return at most ten traces, and for each trace skip the first forty events and return at most twenty events.

```
limit 1:5, t:10, e:20  
offset 1:10, t:20, e:40
```

A.9. Comments

The comment is a sequence of characters that is not interpreted as PQL code. The comments are intended for code documentation. PQL supports the C-style inline comments `// <comment>` and the SQL-style inline comments `-- <comment>` that begin from a comment prefix and ends at the closest newline character or end of input. PQL also supports the C-style block comments `/* <comment> */`. The block comments may span several lines or a part of a single line.

A.10. Known limitations

PQL currently does not support some features of the IEEE 1849-2016 Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams [4]. These features are subject to implementation in a future version of PQL. This section summarizes the unsupported features.

A.10.1. Nested attributes

The IEEE 1849-2016 standard [4] defines *nested attributes* as an optional feature for implementing software, not required for compliance with the IEEE 1849-2016 standard. In the IEEE 1849-2016 standard, the attributes may form a tree, where every node represents a single attribute and the root attribute is attached to a log, a trace, or an event.

PQL does not support references to the nested attributes. However, the reference implementation of PQL supports selecting the entire tree of attributes wherever the tree root attribute is selected. The reference implementation of PQL does not use or interpret the values of the nested attributes.

A.10.2. List attribute

The IEEE 1849-2016 standard [4] defines the *list attribute* as an attribute which value is a list of other attributes.

PQL does not support references to the elements of the list attribute, however, the list attribute itself can be referenced by name. The reference implementation of PQL supports retrieval of the list attribute and its elements wherever the list attribute is referenced in the `select` clause. In all other clauses, the value of the list attribute evaluates to `null`.

Appendix B. PQL grammar

We define the grammar of PQL in the Extended Backus-Naur Format (EBNF) with the ANTLR4 extensions [49]. The grammar splits into two parts: the lexer grammar that defines tokens and the parser grammar that defines parse rules involving these tokens.

```
lexer grammar QLLexer;
// Lexer rules start with uppercase letters
tokens {STRING}
@lexer::members {
    private void go(int diff) {
        _input.seek(_input.index() + diff);
    }
}
```

```

    }
}

SELECT      : 'select' ;
DELETE      : 'delete' ;
WHERE       : 'where' ;
GROUP_BY    : 'group' [ \t\r\n]+ 'by' ;
ORDER_BY    : 'order' [ \t\r\n]+ 'by' ;
LIMIT      : 'limit' ;
OFFSET      : 'offset' ;

SCOPE       : 'log'
            | 'l'
            | 'trace'
            | 't'
            | 'event'
            | 'e'
            ;

UUID        : SCOPE_PREFIX HEX_4 HEX_4 '-' HEX_4 '-' HEX_4 '-' HEX_4 '-'
            HEX_4 HEX_4 HEX_4 ;

STRING_SINGLE : SCOPE_PREFIX '\'' ( '\\\\' | . ) * ? '\'' -> type(String) ;
STRING_DOUBLE : SCOPE_PREFIX '"' ( '\\"' | . ) * ? '"' -> type(String) ;

NUMBER      : SCOPE_PREFIX '-' ? INT ( '.' [0-9] + ) ? EXP ? ;
BOOLEAN     : SCOPE_PREFIX 'true' | 'false' ;
DATETIME    : SCOPE_PREFIX 'D' ISODATE ( 'T' ISOTIME ISOTIMEZONE ? ) ?
            | SCOPE_PREFIX 'D' ISODATESHORT ( 'T' ? ISOTIMESHORT
            ISOTIMEZONE ? ) ?
            ;

NULL        : SCOPE_PREFIX 'null' ;

FUNC_AGGR   : SCOPE_PREFIX ( 'min' | 'max' | 'avg' | 'count' | 'sum' ) '('
            { go(-1); } ;
FUNC_SCALAR1 : SCOPE_PREFIX ( 'date' | 'time' | 'year' | 'month' | 'day' |
            'hour' | 'minute' | 'second' | 'millisecond'
            |
            'quarter' | 'dayofweek' | 'upper' | 'lower' |
            'round' ) '(' { go(-1); } ;
FUNC_SCALAR0 : SCOPE_PREFIX 'now' '(' { go(-1); } ;

OP_MUL      : '*' ;

```

```

OP_DIV      : '/' ;
OP_ADD      : '+' ;
OP_SUB      : '-' ;

OP_LT       : '<' ;
OP_LE       : '<=' ;
OP_GT       : '>' ;
OP_GE       : '>=' ;
OP_EQ       : '=' ;
OP_NEQ      : '!=' ;
;

OP_IS_NULL  : 'is' [ \t\r\n]+ 'null' ;
OP_IS_NOT_NULL: 'is' [ \t\r\n]+ 'not' [ \t\r\n]+ 'null' ;

OP_IN       : 'in' ;
OP_NOT_IN   : 'not' [ \t\r\n]+ 'in' ;

OP_AND      : 'and' ;
OP_OR       : 'or' ;
OP_NOT      : 'not' ;

OP_MATCHES  : 'matches' ;
OP_LIKE     : 'like' ;

L_PARENTHESIS : '(' ;
R_PARENTHESIS : ')' ;
COMMA        : ',' ;
COLON        : ':' ;
ORDER_ASC    : 'asc' ;
ORDER_DESC   : 'desc' ;

ID          : SCOPE_PREFIX (LETTER|DIGIT)+ (':' (LETTER|DIGIT)+)?
           | '[' SCOPE_PREFIX .+? ']' ;

LINE_COMMENT: ( '//' | '--' ) .*? ( '\r' | '\n' ) -> skip ;
COMMENT     : '/*' .*? '*/' -> skip ;
WS          : [ \t\r\n]+ -> skip ; // skip spaces, tabs, newlines, \r (
    Windows)

fragment HOISTING_PREFIX: '^' ;
fragment SCOPE_PREFIX: 'log' ':'
    | 'l' ':'
    | HOISTING_PREFIX? 'trace' ':'
    | HOISTING_PREFIX? 't' ':'
    | HOISTING_PREFIX? HOISTING_PREFIX? 'event' ':'

```

```

        | HOISTING_PREFIX? HOISTING_PREFIX? 'e' ','
        | HOISTING_PREFIX? HOISTING_PREFIX?
    ;

fragment INT      : '0' | [1-9] DIGIT* ;
fragment EXP      : [Ee] [+|-]? INT ;
fragment LETTER   : [a-zA-Z\u0080-\u00FF_] ;
fragment DIGIT    : [0-9] ;
fragment ISODATE: ISOYEAR '-' ISOMONTH '-' ISODAY
        | ISOYEAR '-' ISOMONTH
    ;

fragment ISODATESHORT: ISOYEAR ISOMONTH ISODAY;
fragment ISOTIME: ISOHOUR ':' ISOMINUTE (':' ISOSECOND (':' ISOMILLI)?)?;
fragment ISOTIMESHORT: ISOHOUR ISOMINUTE (ISOSECOND (':' ISOMILLI)?)? ;
fragment ISOYEAR: DIGIT DIGIT DIGIT ;
fragment ISOMONTH: '0' [1-9] | '10' | '11' | '12' ;
fragment ISODAY: '0' [1-9] | [1-2] DIGIT | '30' | '31' ;
fragment ISOHOUR: [0-1] DIGIT | '2' [0-3] ;
fragment ISOMINUTE: [0-5] DIGIT ;
fragment ISOSECOND: ISOMINUTE ;
fragment ISOMILLI: DIGIT DIGIT DIGIT ;
fragment ISOTIMEZONE: 'Z' | ('+'|'-') ISOTZONEHOUR (':' ISOMINUTE)? ;
fragment ISOTZONEHOUR: '0' DIGIT | '10' | '11' | '12' ;
fragment HEX_4: HEX HEX HEX HEX;
fragment HEX: [0-9a-fA-F];

parser grammar QLParse;
options { tokenVocab=QLLexer; }
// Parser rules start with lowercase letters
// Axiom.
query      : read_query EOF
        | delete_query EOF
    ;

read_query : select where? group_by? order_by? limit? offset?
    ;

delete_query: delete where? order_by? limit? offset?
    ;

select      :
        | SELECT '*' (',' column_list)?
        | SELECT column_list
    ;

delete      : DELETE SCOPE? ;

```

```

where      : WHERE logic_expr ;

group_by   : GROUP_BY id_list ;

order_by   : ORDER_BY column_list_with_order ;

limit      : LIMIT limit_number (',' limit_number)*;

offset     : OFFSET offset_number (',' offset_number)*;

column_list : SCOPE COLON '*'
            | SCOPE COLON '*' ',' column_list
            | arith_expr_root

            | arith_expr_root ',' column_list

            ;

arith_expr_root : arith_expr ;

id_list       : ID (',' ID)* ;

column_list_with_order : ordered_expression_root (','
            ordered_expression_root)* ;

ordered_expression_root : arith_expr order_dir ;

order_dir    :
            | ORDER_ASC
            | ORDER_DESC
            ;

limit_number: NUMBER ;
offset_number: NUMBER ;

scalar      : STRING
            | NUMBER
            | BOOLEAN
            | DATETIME
            | UUID
            | NULL
            ;

// The order of productions reflects the operator precedence
arith_expr  : '(' arith_expr ')'
```

```

    | arith_expr ('*' | '/') arith_expr
    | arith_expr ('+' | '-') arith_expr
    | func
    | ID
    | scalar
    ;

// Note: Aggregation functions can only take a column identifier as an
//       argument
func      : FUNC_SCALAR0 '(' ')'
          | FUNC_SCALAR1 '(' arith_expr ')'
          | FUNC_AGGR '(' ID ')'
          ;

// The order of productions reflects the operator precedence
logic_expr : '(' logic_expr ')'
          | arith_expr (OP_IN | OP_NOT_IN) in_list
          | arith_expr (OP_MATCHES | OP_LIKE) STRING
          | arith_expr (OP_LT | OP_LE | OP_EQ | OP_NEQ | OP_GT | OP_GE)
            arith_expr
          | arith_expr (OP_IS_NULL | OP_IS_NOT_NULL)
          | OP_NOT logic_expr
          | logic_expr OP_AND logic_expr
          | logic_expr OP_OR logic_expr
          ;

in_list   : '(' id_or_scalar_list ')' ;

id_or_scalar_list : (ID | scalar) (',' (ID | scalar))* ;

```